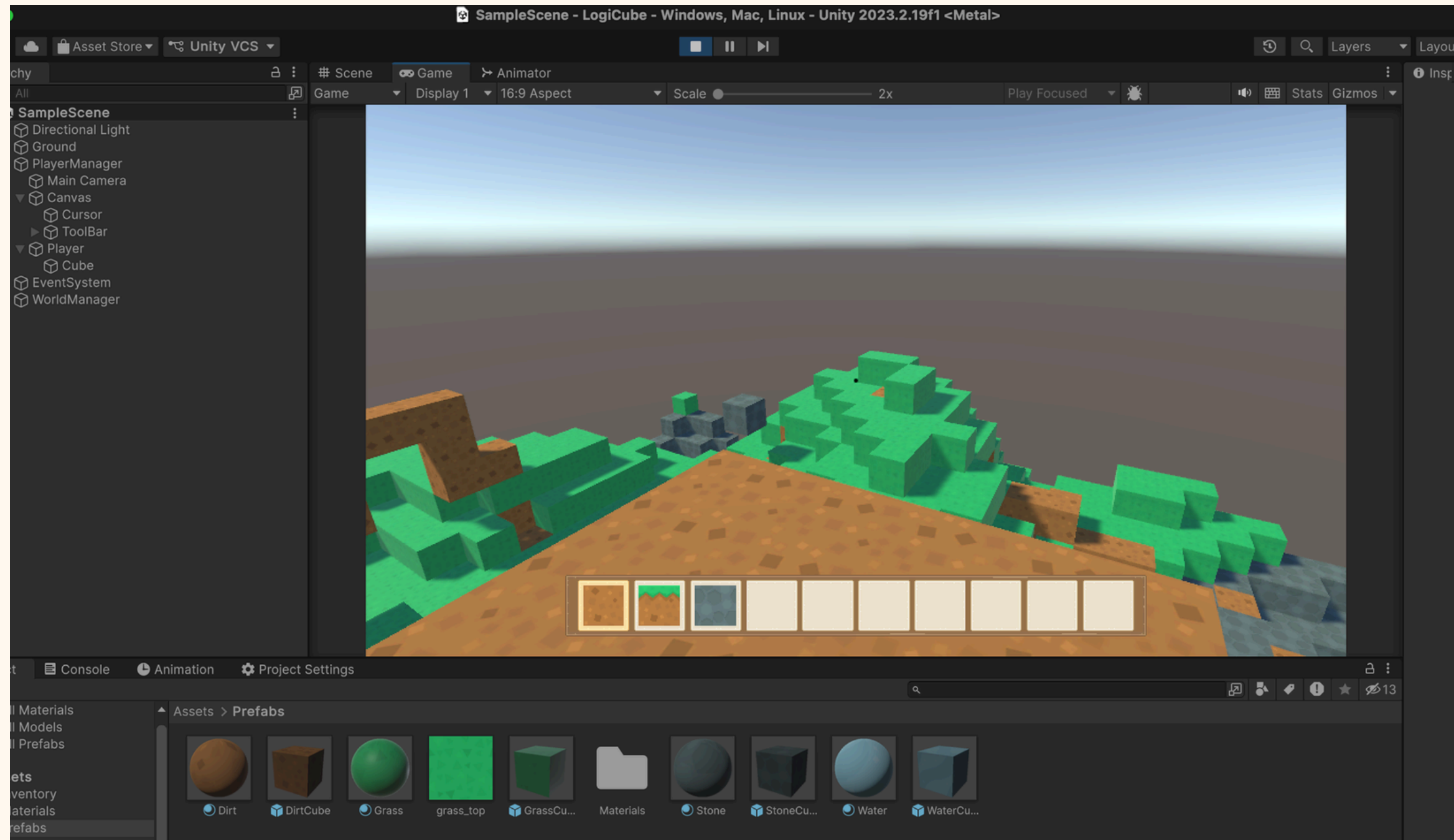


LogiCube

Personnage

Objectif : créer un joueur capable de marcher sur notre mesh



Première étape : ajouter un collider à notre mesh

```
private void RebuildChunk()
{
    var visibleFaces = new Dictionary<Vector3Int, List<FaceDirection>>();

    foreach (var (pos, type) in blocks)
    {
        if (!type.IsSolid()) continue;

        var faces = FaceCuller.GetVisibleFaces(pos, blocks);
        if (faces.Count > 0)
            visibleFaces[pos] = faces;
    }

    Mesh mesh = ChunkMeshBuilder.Build(visibleFaces, blocks, out var subMeshOrder);

    // Create chunk object once
    if (chunkGO == null)
    {
        chunkGO = new GameObject("Chunk");
        chunkGO.transform.SetParent(transform);

        chunkGO.AddComponent<MeshFilter>();
        chunkGO.AddComponent<MeshRenderer>();
        chunkGO.AddComponent<MeshCollider>();
    }

    // Assign mesh
    chunkGO.GetComponent<MeshFilter>().mesh = mesh;
}
```

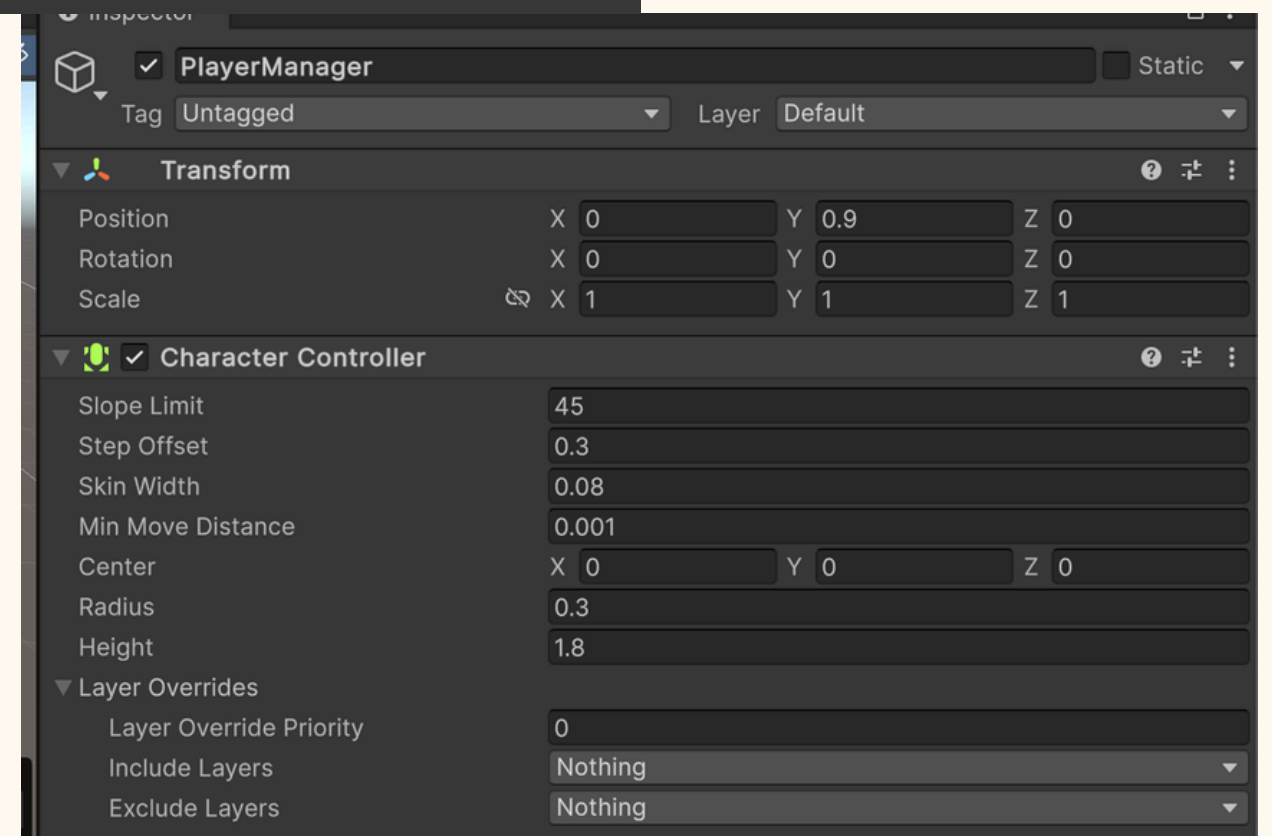
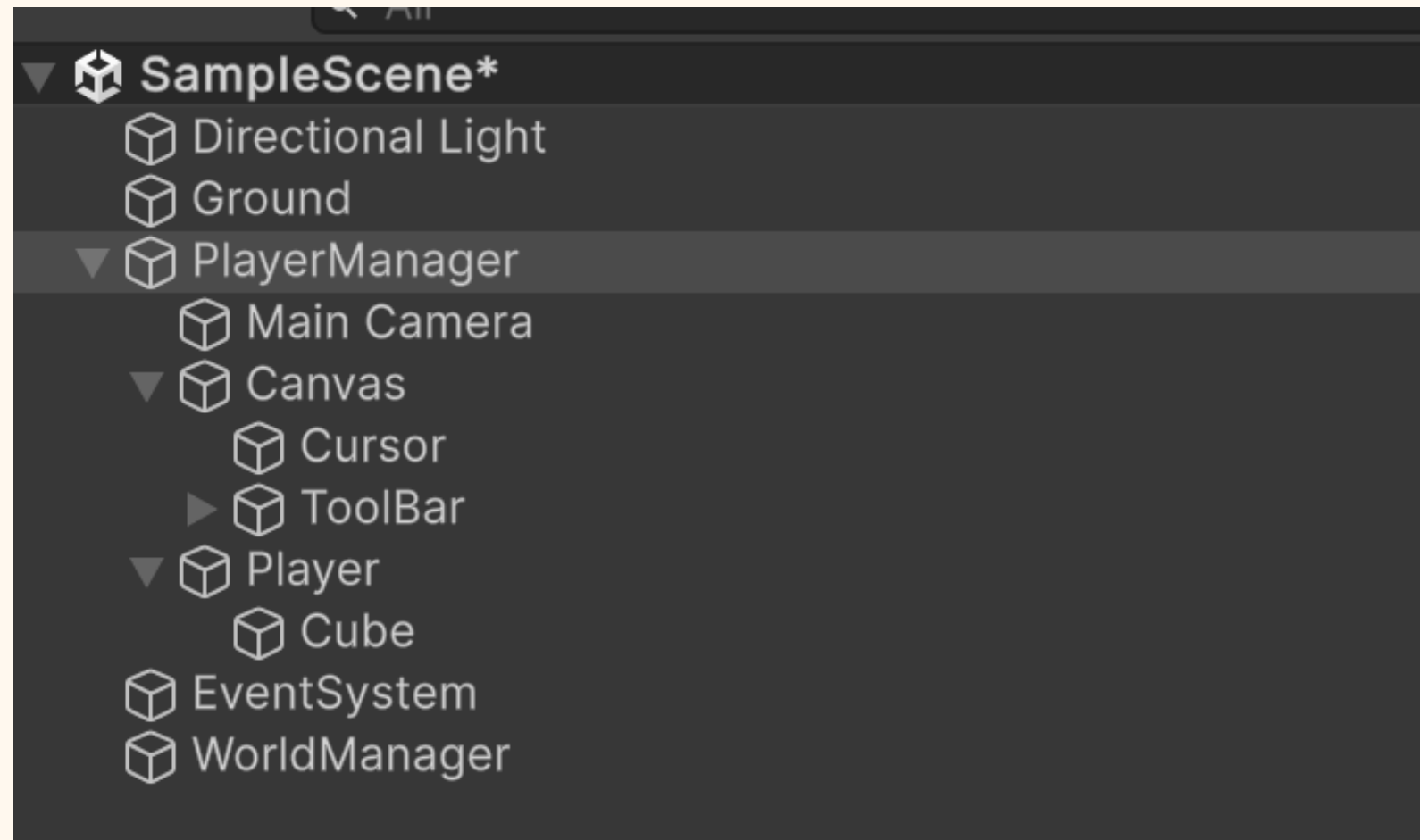
- Dans le script WorldManagerScript, dans la fonction RebuildChunk, on ajoute ce code pour attacher un MeshCollider, un MeshRenderer et un MeshFilter au GameObject du chunk.

Configuration de PlayerManager

Sur PlayerManager, ajoutez un CharacterController.

Réglages conseillés :

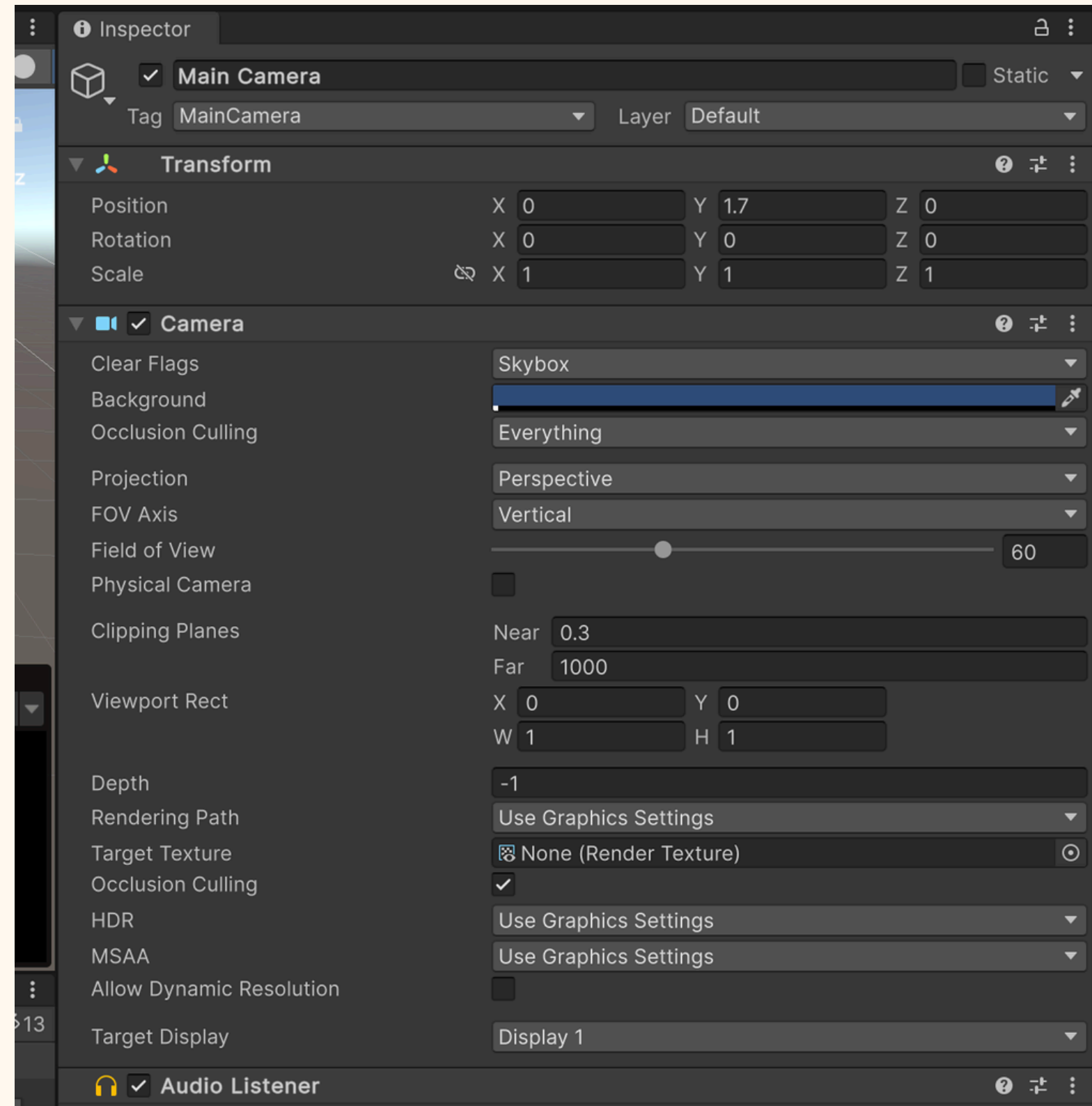
- Center = (0, 0.9, 0)
- Height = 1.8
- Radius = 0.3
- Step Offset = 0.3



Configuration de la caméra

On place la Main Camera comme enfant de PlayerManager pour qu'elle suive le joueur.
Sa position est :

- $X = 0$
- $Y = 1.62$
- $Z = 0$



Remplacer FlyCamera par un script de joueur

- FlyCamera n'est plus adapté à notre joueur
- on le remplace par un script FirstPersonPlayer
- ce script est ajouté à PlayerManager
- il contient les variables utiles pour marcher, sauter et regarder autour
- ajoutez ces variables d'instances

```
public class FirstPersonPlayer : MonoBehaviour
{
    [Header("Déplacements")]
    public float moveSpeed = 5f;           // vitesse pour marcher
    public float flySpeed = 7f;            // vitesse pour voler
    public float lookSpeed = 2f;           // vitesse de la souris
    public float gravity = -20f;            // force qui fait tomber
    public float jumpHeight = 1.2f;        // hauteur du saut
    public float doubleTapDelay = 0.3f;    // temps max entre 2 appuis sur espace

    [Header("Références")]
    public Transform cameraTransform;      // la caméra du joueur

    private CharacterController controller;

    private float pitch = 0f;              // rotation verticale de la caméra
    private float verticalVelocity = 0f;    // vitesse vers le haut ou vers le bas

    private bool isFlying = false;         // est-ce que le joueur vole ?
    private float lastSpacePressTime = -10f; // moment du dernier appui sur espace
```

FirstPersonPlayer : fonctions principales

FirstPersonPlayer : fonctions essentielles

- Look() gère la rotation du joueur et de la caméra
- HandleJumpAndFlyToggle() gère le saut et le mode vol
- le script complet peut être consulté sur GitHub

lien vers le script:

<https://github.com/Ayly-EXE/LogiCube/blob/Cours4/PlayerInventory/Assets/Scripts/FirstPersonPlayer.cs>

```
void Look()
{
    // On lit le mouvement de la souris
    float mouseX = Input.GetAxis("Mouse X") * lookSpeed;
    float mouseY = Input.GetAxis("Mouse Y") * lookSpeed;

    // Tourner à gauche/droite = tourner le joueur entier
    transform.Rotate(0f, mouseX, 0f, Space.Self);

    // On enlève tout penchement bizarre du corps
    Vector3 bodyEuler = transform.eulerAngles;
    transform.eulerAngles = new Vector3(0f, bodyEuler.y, 0f);

    // Regarder en haut/bas = tourner seulement la caméra
    pitch -= mouseY;
    pitch = Mathf.Clamp(pitch, -89f, 89f); // limite pour ne pas tourner complètement
    cameraTransform.localRotation = Quaternion.Euler(pitch, 0f, 0f);
}
```

```
void HandleJumpAndFlyToggle()
{
    // Si on n'appuie pas sur espace, on ne fait rien
    if (!Input.GetKeyDown(KeyCode.Space))
        return;

    float now = Time.time;

    // Vérifie si on a appuyé 2 fois rapidement sur espace
    bool isDoubleTap = now - lastSpacePressTime <= doubleTapDelay;
    lastSpacePressTime = now;

    // Si on est dans les airs et qu'on fait un double appui :
    // on active ou désactive le mode vol
    if (!controller.isGrounded && isDoubleTap)
    {
        isFlying = !isFlying;

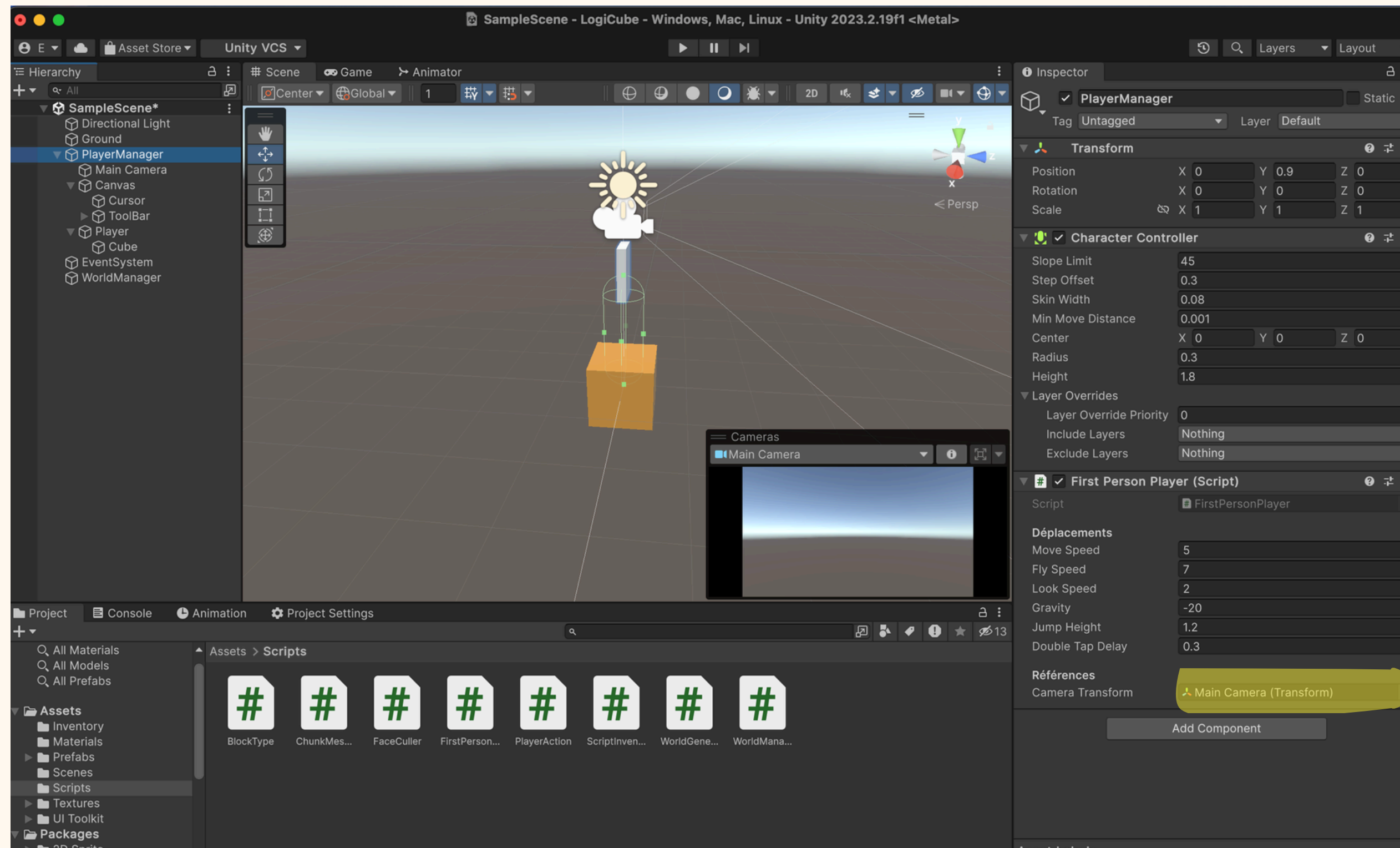
        // Si on commence à voler, on arrête la chute
        if (isFlying)
        {
            verticalVelocity = 0f;
        }

        return;
    }

    // Saut normal : seulement si on est au sol et qu'on ne vole pas
    if (controller.isGrounded && !isFlying)
    {
        verticalVelocity = Mathf.Sqrt(jumpHeight * -2f * gravity);
    }
}
```

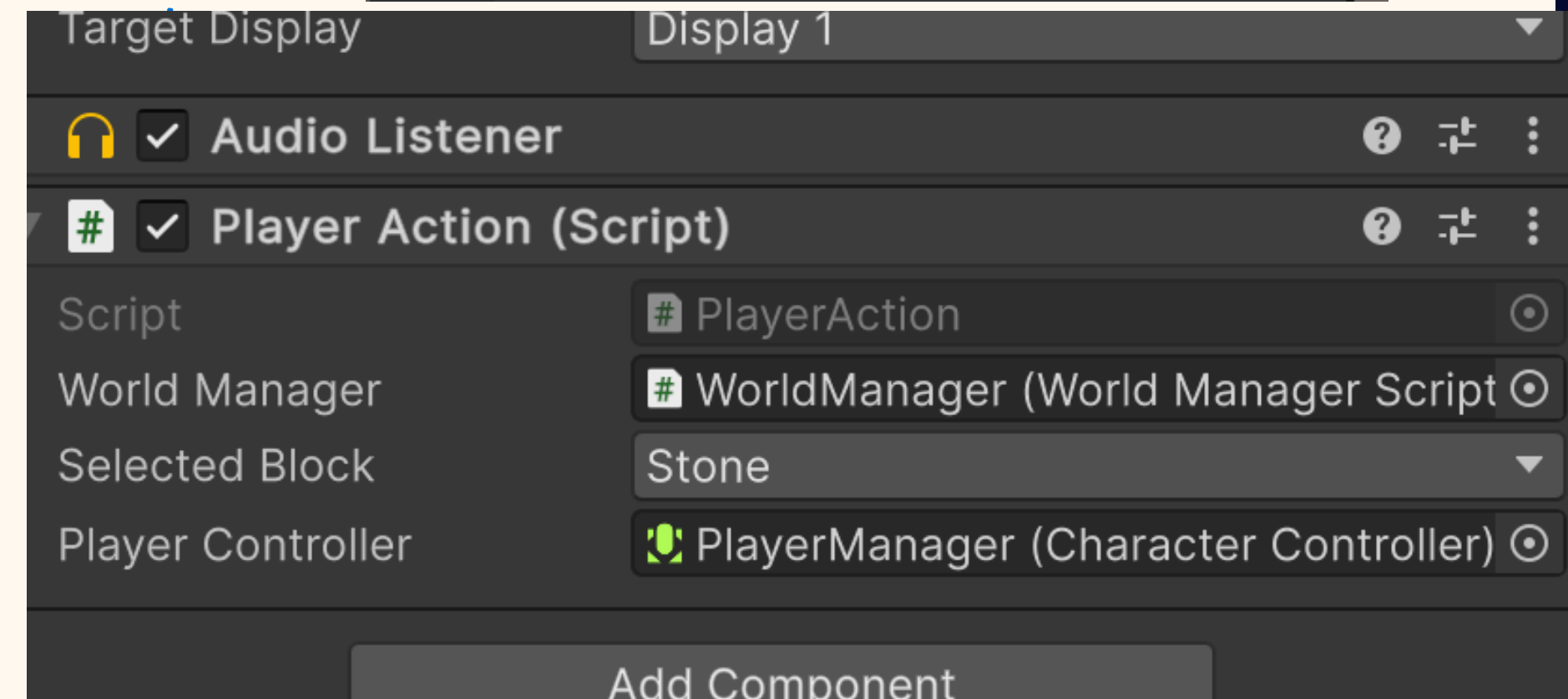
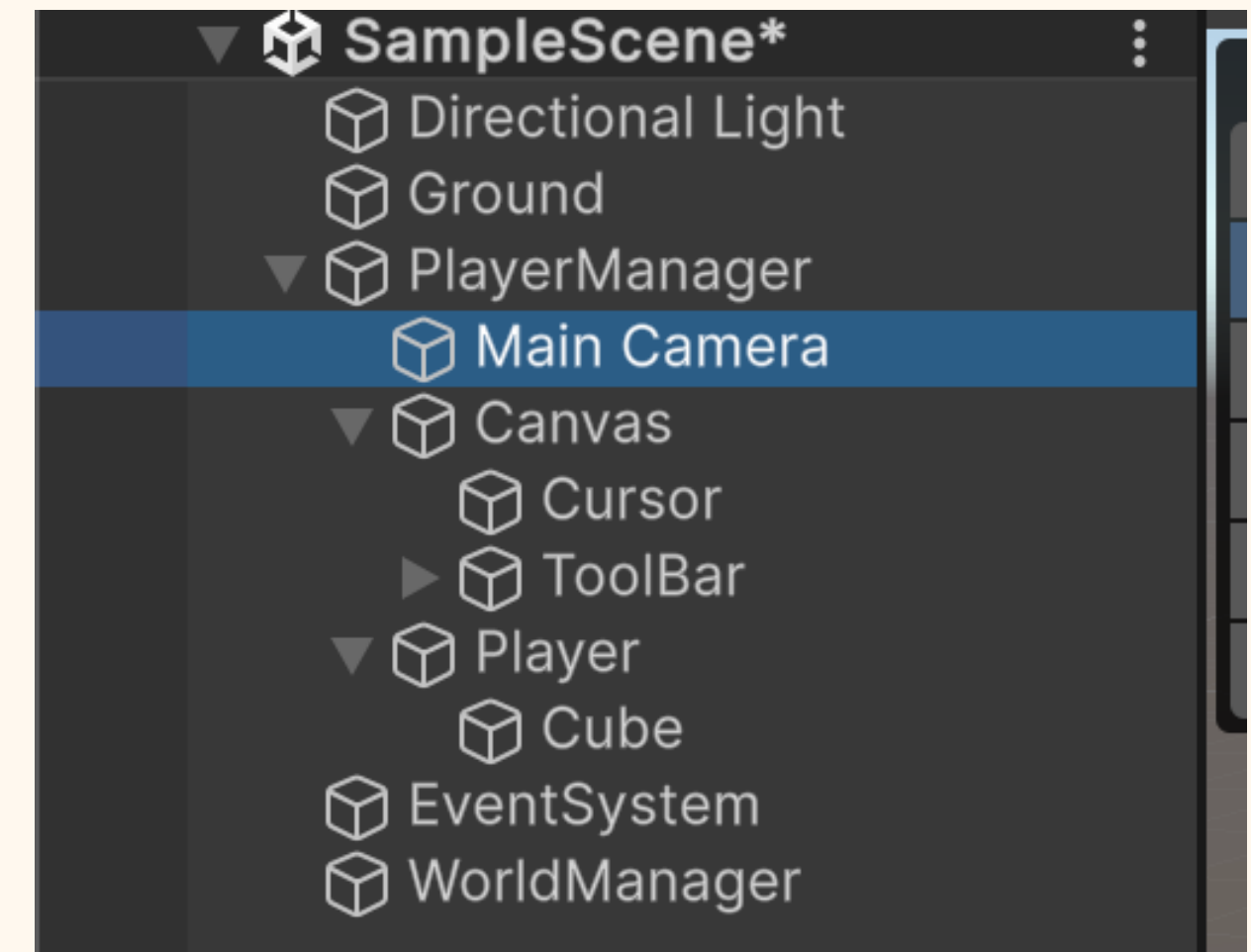
Configurer FirstPersonPlayer dans Unity

- Dans Unity
- sur PlayerManager, ajouter le script FirstPersonPlayer
- glisser la Main Camera dans le champ cameraTransform



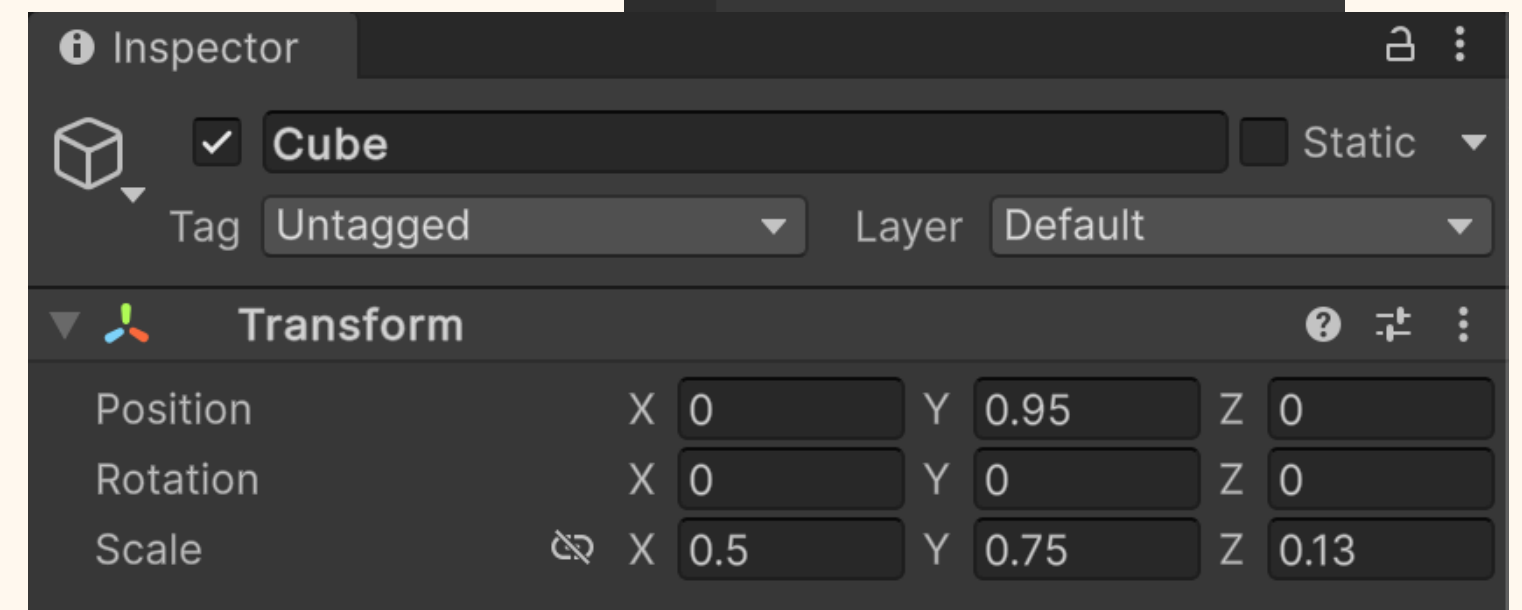
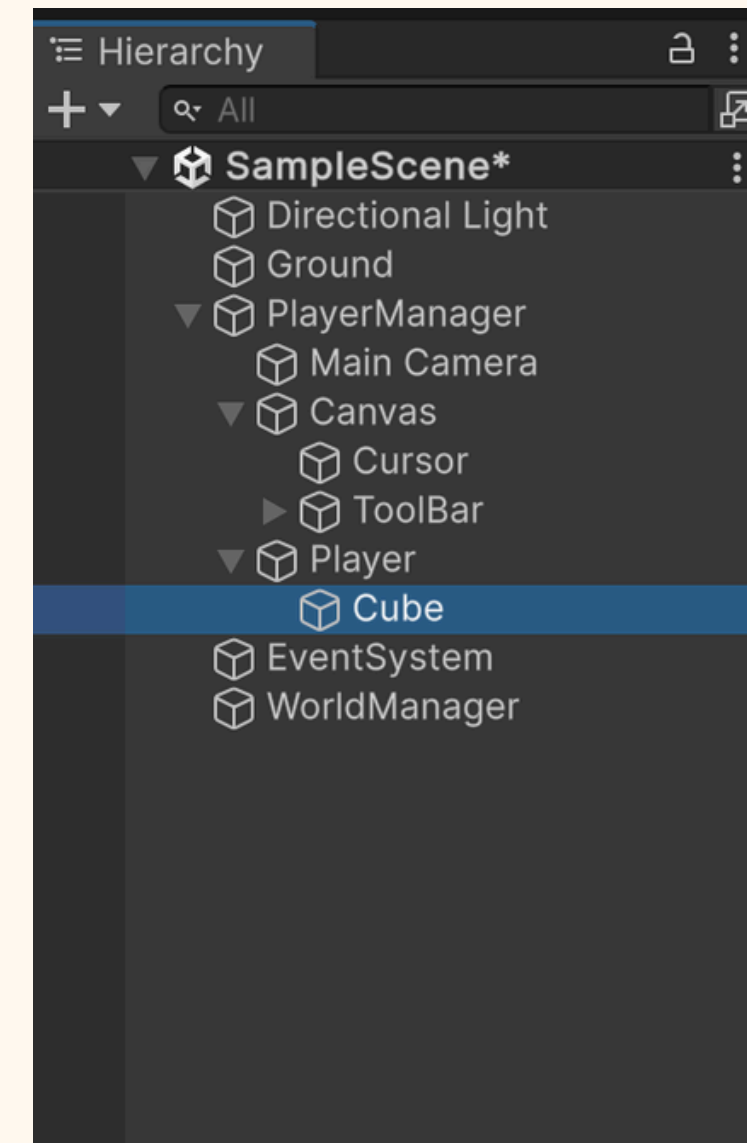
Configurer PlayerAction sur la caméra

- **Ajouter PlayerAction à la Main Camera**
 - dans Unity, sélectionner la Main Camera
 - ajouter le script PlayerAction



Créer un personnage simple

- pour commencer, on utilise un personnage très simple : un cube
- sous **PlayerManager**, créer un objet vide nommé **Player**
- ajouter un **Cube** comme enfant de **Player**
- appliquer les paramètres affichés



Mettre à jour PlayerAction

- Dans PlayerAction.cs :
- clic gauche : casser un bloc
- clic droit : placer un bloc
- rayon depuis la caméra pour viser
- on détecte que le raycast ne touche pas le joueur

```
bool IsPlayerObject(Transform hitTransform)
{
    if (playerController == null)
        return false;

    Transform playerRoot = playerController.transform;
    return hitTransform == playerRoot || hitTransform.IsChildOf(playerRoot);
}
```

```
void Update()
{
    if (Input.GetMouseButtonDown(1))
        TryPlaceBlock();

    if (Input.GetMouseButtonDown(0))
        TryBreakBlock();
}
```

```
// Envoie un rayon devant la caméra pour voir ce qu'on vise
2 references
RaycastHit? GetHit(float maxDistance = 100f)
{
    // On démarre un tout petit peu devant la caméra
    // pour éviter de toucher le joueur directement
    Vector3 origin = transform.position + transform.forward * 0.1f;
    Ray ray = new Ray(origin, transform.forward);

    // On récupère tous les objets touchés par le rayon
    RaycastHit[] hits = Physics.RaycastAll(ray, maxDistance);

    // On trie les objets du plus proche au plus loin
    System.Array.Sort(hits, (a, b) => a.distance.CompareTo(b.distance));

    // On cherche le premier objet qui n'est pas le joueur
    foreach (RaycastHit hit in hits)
    {
        if (IsPlayerObject(hit.transform))
            continue;

        return hit;
    }

    // Rien n'a été touché
    return null;
}
```

Mettre à jour PlayerAction

```
1 reference
void TryPlaceBlock()
{
    RaycastHit? hit = GetHit();
    if (!hit.HasValue)
        return;

    Vector3Int placePos = GetBlockPlacementPosition(hit.Value);

    if (!IsInsidePlayer(placePos))
        worldManager.PlaceBlock(placePos, selectedBlock);
}

1 reference
void TryBreakBlock()
{
    RaycastHit? hit = GetHit();
    if (!hit.HasValue)
        return;

    Vector3Int hitBlockPos = GetHitBlockPosition(hit.Value);
    worldManager.DestroyBlock(hitBlockPos);
}
```

```
// Vérifie si le bloc qu'on veut poser serait dans le joueur
bool IsInsidePlayer(Vector3Int blockPos)
{
    if (playerController == null)
        return false;

    // On prend la boîte du joueur
    Bounds b = playerController.bounds;

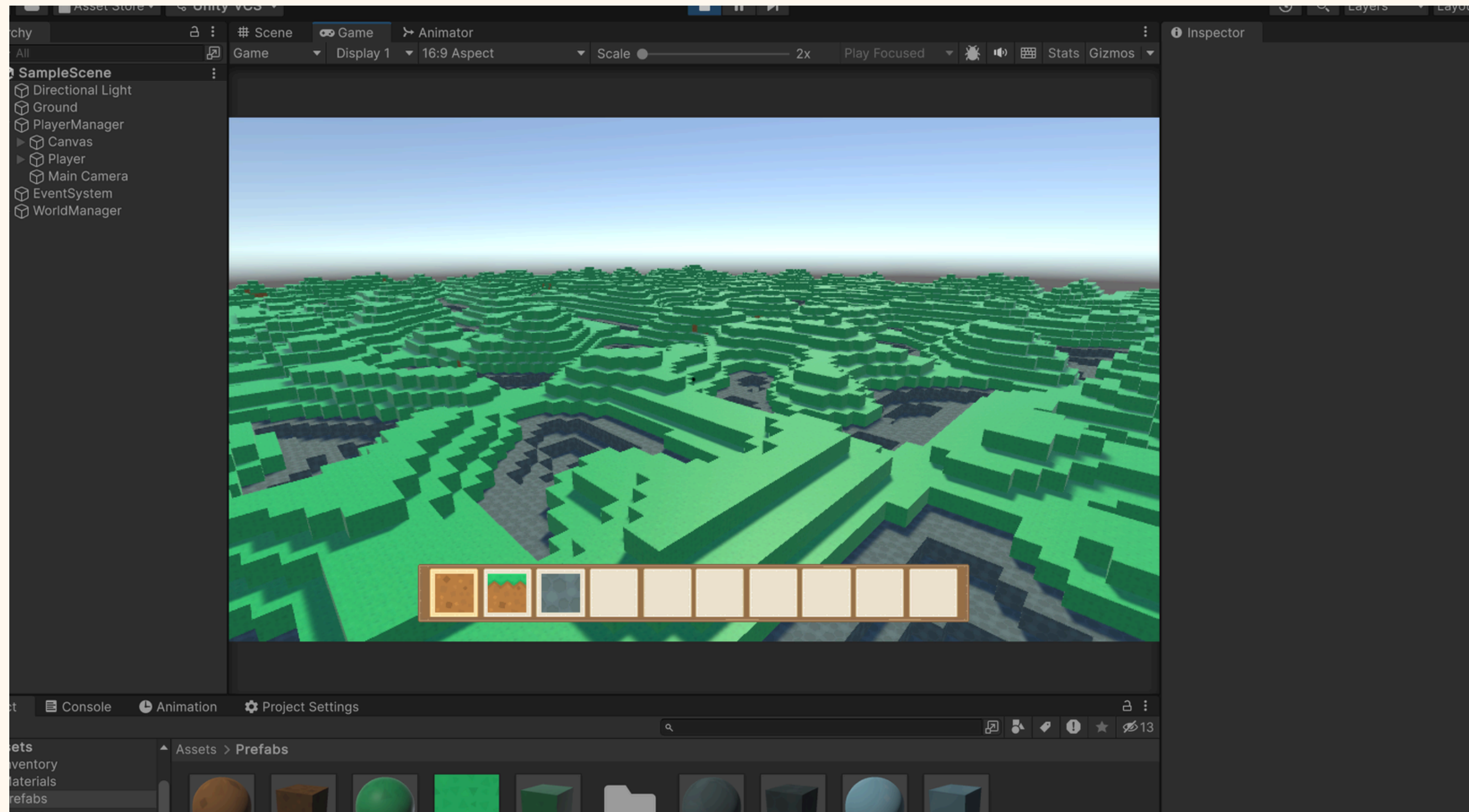
    // On récupère les coins min et max de cette boîte
    // avec une toute petite marge pour éviter les bords exacts
    Vector3 min = b.min + Vector3.one * 0.001f;
    Vector3 max = b.max - Vector3.one * 0.001f;

    // On parcourt toutes les cases occupées par le joueur
    // entre le coin minimum et le coin maximum
    for (int x = Mathf.FloorToInt(min.x); x <= Mathf.FloorToInt(max.x); x++)
    {
        for (int y = Mathf.FloorToInt(min.y); y <= Mathf.FloorToInt(max.y); y++)
        {
            for (int z = Mathf.FloorToInt(min.z); z <= Mathf.FloorToInt(max.z); z++)
            {
                // Si la case du bloc à poser est une case du joueur,
                // alors on ne peut pas poser le bloc
                if (new Vector3Int(x, y, z) == blockPos)
                    return true;
            }
        }
    }
}
```

script complet:

<https://github.com/Ayly-EXE/LogiCube/blob/Cours4/PlayerInventory/Assets/Scripts/PlayerAction.cs>

Plusieurs chunks !



Dans votre version actuel,
WorldManagerScript.cs faisait presque tout :

- charger la seed
- générer le monde
- construire le mesh
- gérer les blocs cassés / posés
- sauvegarder

=> C'était pratique au début, mais ce script
faisait trop de choses à la fois.

```
//  
void Start()  
{  
    // Build fast lookup dictionary  
    materialLookup.Clear();  
    foreach (var entry in blockMaterials)  
        materialLookup[entry.type] = entry.material;  
  
    var save = LoadOrCreate();  
    seed = save.seed;  
  
    // 1) On génère le monde de base avec la seed  
    blocks = generator.Generate(seed);  
  
    // 2) On remet les modifications du joueur (blocs posés/détruits)  
    ApplyDelta(save);  
  
    // 3) On construit le rendu 3D  
    RebuildChunk();  
}
```

```
// 4) On replace  
if (save.player  
{  
    player.trans  
    player.trans  
}  
}  
  
public void PlaceBlock(Vector3 worldPos, BlockType type)  
{  
    var p = Vector3Int.FloorToInt(worldPos);  
    blocks[p] = type;  
    RebuildChunk();  
}
```

```
public void DestroyBlock(Vector3 worldPos)  
{  
    var p = Vector3Int.FloorToInt(worldPos);  
    if (!blocks.ContainsKey(p)) return;  
}
```

```
private void RebuildChunk()  
{  
    var visibleFaces = new Dictionary<Vector3Int, List<FaceDirection>>();  
  
    foreach (var (pos, type) in blocks)  
    {  
        if (!type.IsSolid()) continue;  
  
        var faces = FaceCuller.GetVisibleFaces(pos, blocks);  
        if (faces.Count > 0)  
            visibleFaces[pos] = faces;  
    }  
  
    Mesh mesh = ChunkMeshBuilder.Build(visibleFaces, blocks, out var subMeshOrder);  
  
    // Create chunk object once  
    if (chunkGO == null)  
    {
```

ChunkSpawner

On va créer un nouveau script, ChunkSpawner.cs, dans lequel on déplacera une partie de la logique de WorldManager.cs, afin de mieux séparer les responsabilités.

Objectif :

- **WorldManager : gérer le monde du côté du joueur**
- **ChunkSpawner : gérer quels chunks doivent apparaître**

Créer un nouveau script ChunkSpawner sur Unity

Etape 1

- Dans un premier temps, on va enlever une partie de la logique du WorldManager pour la déléguer au ChunkSpawner.
- Enlever les variables d'instance seed et chunkGO.
- Ajouter une référence vers votre nouveau script chunkSpawner

Avant

```
public class WorldManagerScript : MonoBehaviour
{
    [Header("Références")]
    public WorldGenerator generator;
    public GameObject player;

    [Header("Matériaux")]
    public List<BlockMaterialEntry> blockMaterials = new();

    private Dictionary<BlockType, Material> materialLookup = new();

    private Dictionary<Vector3Int, BlockType> blocks = new();
    private int seed;

    private GameObject chunkGO;

    private string SavePath => Path.Combine(Application.persistentDataPath, "save.json");
```

Après

```
[Header("Références")]
17 references | ☑ Unity Serialized Field
public WorldGenerator generator;
7 references | ☑ Unity Serialized Field
public GameObject player;
6 references | ☑ Unity Serialized Field
public ChunkSpawner chunkSpawner;

[Header("Matériaux")]
1 reference | ☑ Unity Serialized Field
public List<BlockMaterialEntry> blockMaterials = new();

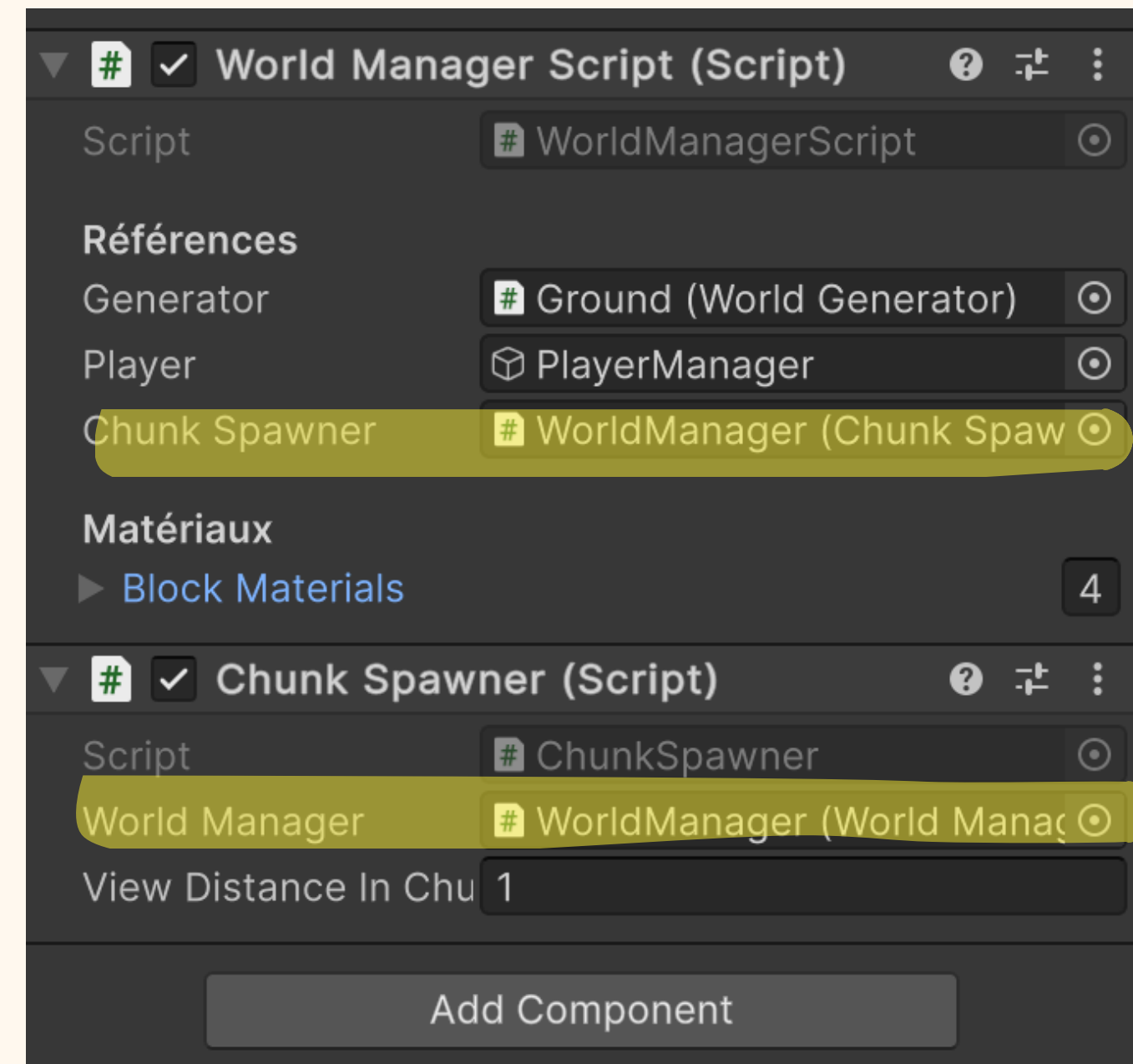
3 references
private Dictionary<BlockType, Material> materialLookup = new();

6 references
private Dictionary<Vector3Int, BlockType> blocks = new();

2 references
private string SavePath => Path.Combine(Application.persistentDataPath, "save.json");
8 references
public Dictionary<Vector3Int, BlockType> Blocks => blocks;
```

- Ajouter également une référence dans le ChunkSpawner vers le WorldManagerScript.
- Ensuite, dans Unity, sur l'objet qui contient le script WorldManager, ajouter aussi le script ChunkSpawner et relier les deux références.

```
1 reference | Unity Script
public class ChunkSpawner : MonoBehaviour
{
    24 references | Unity Serialized Field
    public WorldManagerScript worldManager;
    4 references | Unity Serialized Field
    public int viewDistanceInChunks = 1;
```



- On délègue une partie du travail de Start() à la fonction Initialize() du ChunkSpawner.
- Start() devient plus court, car il ne gère plus directement toute la logique des chunks.

Avant

```
void Start()
{
    // Build fast lookup dictionary
    materialLookup.Clear();
    foreach (var entry in blockMaterials)
        materialLookup[entry.type] = entry.material;

    var save = LoadOrCreate();
    seed = save.seed;

    // 1) On génère le monde de base avec la seed
    blocks = generator.Generate(seed);

    // 2) On remet les modifications du joueur (blocs posés/détruits)
    ApplyDelta(save);

    // 3) On construit le rendu 3D
    RebuildChunk();

    // 4) On replace le joueur où il était
    if (save.player != null && player != null)
    {
        player.transform.position = save.player.position;
        player.transform.rotation = save.player.rotation;
    }
}
```

Après

```
void Start()
{
    materialLookup.Clear();
    foreach (var entry in blockMaterials)
        materialLookup[entry.type] = entry.material;

    var save = chunkSpawner.Initialize(this, SavePath);
    player.transform.position = save.player.position;
    player.transform.rotation = save.player.rotation;
}
```

- PlaceBlock() délègue la gestion spéciale des chunks à chunkSpawner.OnBlockPlaced(...).
- DestroyBlock() délègue la mise à jour des chunks à chunkSpawner.OnBlockDestroyed(...).
- Les fonctions du WorldManager gardent leur logique de base, mais deviennent plus légères.

Avant

```
public void PlaceBlock(Vector3 worldPos, BlockType type)
{
    var p = Vector3Int.FloorToInt(worldPos);
    blocks[p] = type;
    RebuildChunk();
}

public void DestroyBlock(Vector3 worldPos)
{
    var p = Vector3Int.FloorToInt(worldPos);
    if (!blocks.ContainsKey(p)) return;
    blocks.Remove(p);
    RebuildChunk();
}
```

Après

```
1 reference
public void PlaceBlock(Vector3 worldPos, BlockType type)
{
    var p = Vector3Int.FloorToInt(worldPos);
    blocks[p] = type;

    chunkSpawner.OnBlockPlaced(p, type);
}

1 reference
public void DestroyBlock(Vector3 worldPos)
{
    var p = Vector3Int.FloorToInt(worldPos);
    if (!blocks.ContainsKey(p)) return;

    blocks.Remove(p);
    chunkSpawner.OnBlockDestroyed(p);
    RebuildChunk();
}
```

```
public void SaveWorld()
{
    chunkSpawner.SaveWorld(SavePath, player);
}
```

```
public void RebuildChunk()
{
    chunkSpawner.RebuildLoadedChunks();
}
```

- SaveWorld() n'écrit plus lui-même toute la sauvegarde : il demande au ChunkSpawner de le faire.
- RebuildChunk() ne reconstruit plus lui-même le monde : il appelle le script qui gère les chunks.

**Vous pouvez copier-coller le script
ChunkSpawner depuis le GitHub.
Le script est plus long, donc on ne va pas tout
détailler ligne par ligne.
On va surtout expliquer ensemble ses
fonctions principales.**

**[https://github.com/Ayly-
EXE/LogiCube/blob/Cours5/TntAndMultiplesCh
unk/Assets/Scripts/ChunkSpawner.cs](https://github.com/Ayly-EXE/LogiCube/blob/Cours5/TntAndMultiplesChunk/Assets/Scripts/ChunkSpawner.cs)**

- On calcule la zone du chunk avec startX, startZ et chunkSize.
- Pour chaque case (x, z), on calcule la hauteur avec le bruit de Perlin.
- Pour chaque bloc (x, y, z), on choisit son type :
- Stone, Dirt ou Grass.
- On vérifie ensuite si le joueur avait déjà modifié ce bloc :
- blocs placés ou supprimés.
- On ajoute enfin un GameObject Unity pour ce chunk avec :
- MeshFilter, MeshRenderer, MeshCollider.

```

2 references
private void CreateChunk(Vector2Int chunkCoord)
{
    int half = worldManager.generator.chunkSize / 2;
    int startX = chunkCoord.x * worldManager.generator.chunkSize - half;
    int startZ = chunkCoord.y * worldManager.generator.chunkSize - half;
    Vector2 offset = GetSeedOffset();

    for (int x = startX; x < startX + worldManager.generator.chunkSize; x++)
        for (int z = startZ; z < startZ + worldManager.generator.chunkSize; z++)
        {
            int height = GetTerrainHeight(x, z, offset);

            for (int y = 0; y <= height; y++)
            {
                Vector3Int pos = new Vector3Int(x, y, z);

                if (removedBlocks.Contains(pos))
                    continue;

                if (placedBlocks.TryGetValue(pos, out var placedType))
                    worldManager.Blocks[pos] = placedType;
                else
                    worldManager.Blocks[pos] = GetBlockTypeForHeight(y, height);
            }
        }

    foreach (var (pos, type) in placedBlocks)
        if (GetChunkCoord(pos) == chunkCoord)
            worldManager.Blocks[pos] = type;

    GameObject chunkGO = new GameObject($"Chunk_{chunkCoord.x}_{chunkCoord.y}");
    chunkGO.transform.SetParent(transform);
    chunkGO.AddComponent<MeshFilter>();
    chunkGO.AddComponent<MeshRenderer>();
    chunkGO.AddComponent<MeshCollider>();

    chunks[chunkCoord] = chunkGO;
}

```

- On crée une liste `chunksToKeep` avec tous les chunks autour du joueur.
- On utilise `viewDistanceInChunks` pour savoir combien de chunks garder.
- On parcourt les chunks déjà chargés :
- ceux qui ne sont plus dans la zone vont dans `chunksToDelete`.
- On supprime ces chunks avec `DeleteChunk(...)`.
- On ajoute les chunks manquants avec `CreateChunk(...)`.
- On finit par `RebuildLoadedChunks()` pour mettre à jour le rendu.

```
2 references
private void UpdateChunks()
{
    List<Vector2Int> chunksToKeep = new List<Vector2Int>();

    for (int x = -viewDistanceInChunks; x <= viewDistanceInChunks; x++)
        for (int z = -viewDistanceInChunks; z <= viewDistanceInChunks; z++)
            chunksToKeep.Add(new Vector2Int(playerChunk.x + x, playerChunk.y + z));

    List<Vector2Int> chunksToDelete = new List<Vector2Int>();
    foreach (var chunk in chunks.Keys)
        if (!chunksToKeep.Contains(chunk))
            chunksToDelete.Add(chunk);

    foreach (var chunk in chunksToDelete)
        DeleteChunk(chunk);

    foreach (var chunk in chunksToKeep)
        if (!chunks.ContainsKey(chunk))
            CreateChunk(chunk);

    RebuildLoadedChunks();
}
```

- On recalcule la zone du chunk avec startX, startZ et chunkSize.
- On parcourt toutes les positions de ce chunk.
- On enlève ces blocs du dictionnaire worldManager.Blocks.
- On détruit ensuite l'objet Unity du chunk avec Destroy(...).
- Enfin, on retire ce chunk du dictionnaire chunks.

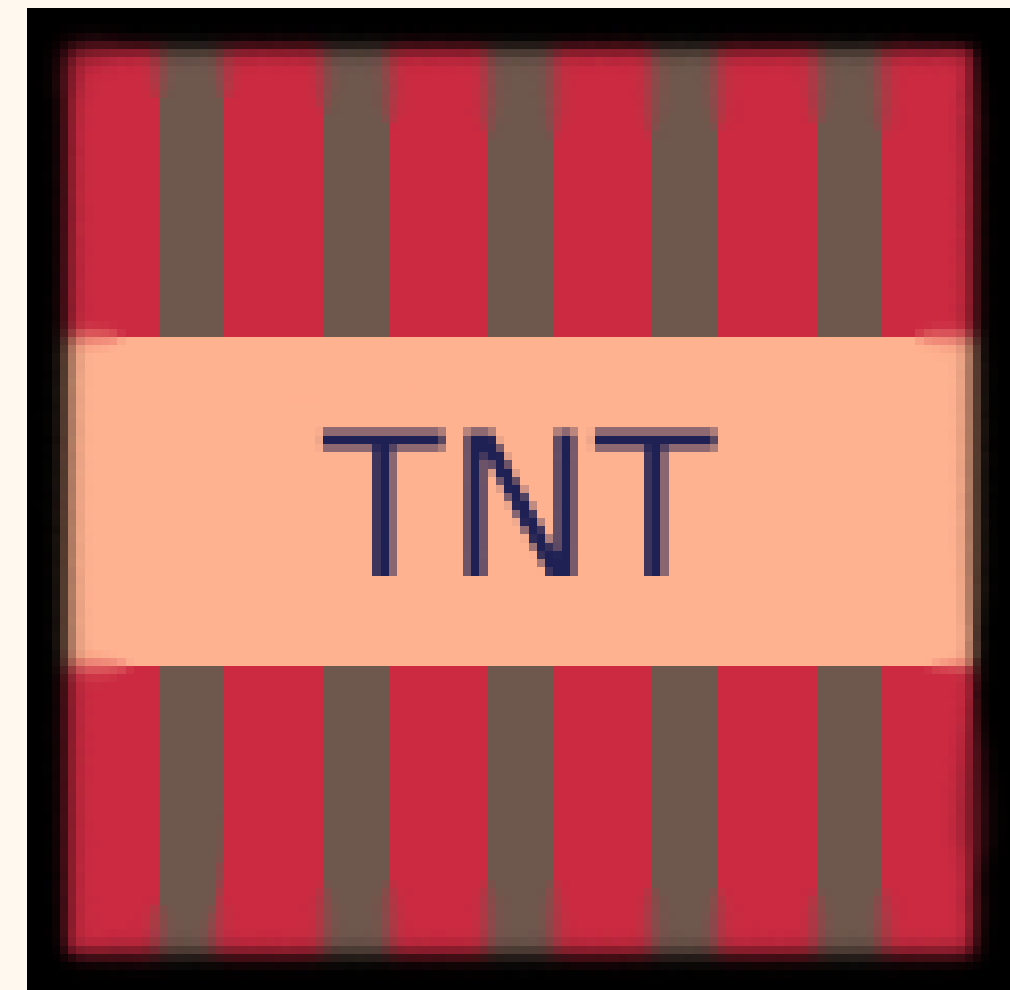
```
void DeleteChunk(Vector2Int chunkCoord)

half = worldManager.generator.chunkSize / 2;
startX = chunkCoord.x * worldManager.generator.chunkSize - half;
startZ = chunkCoord.y * worldManager.generator.chunkSize - half;

for (int x = startX; x < startX + worldManager.generator.chunkSize; x++)
    for (int z = startZ; z < startZ + worldManager.generator.chunkSize; z++)
        for (int y = 0; y <= worldManager.generator.terrainHeight; y++)
            worldManager.Blocks.Remove(new Vector3Int(x, y, z));

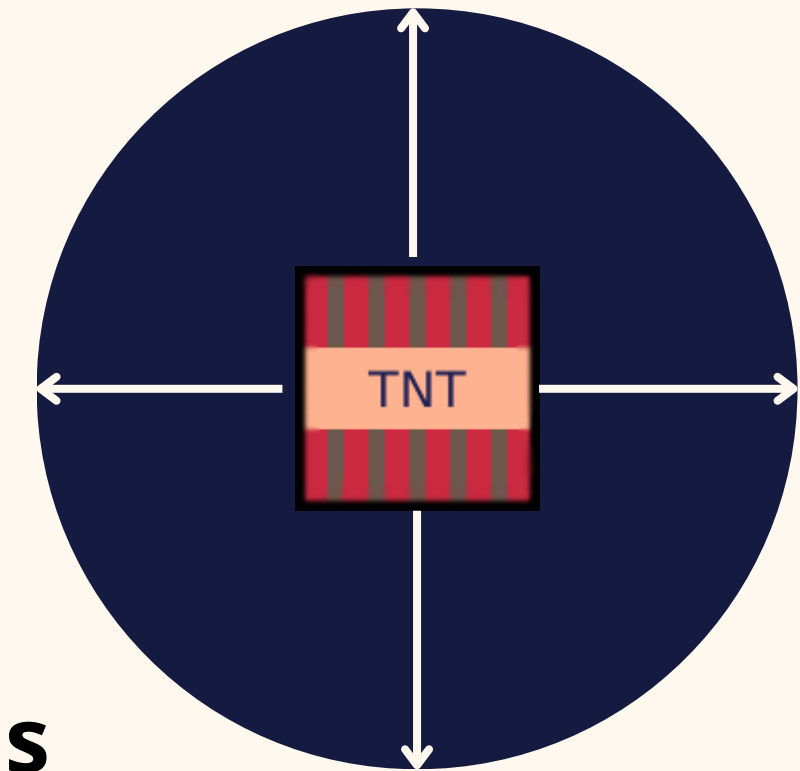
Destroy(chunks[chunkCoord]);
chunks.Remove(chunkCoord);
```

Ça va peter !



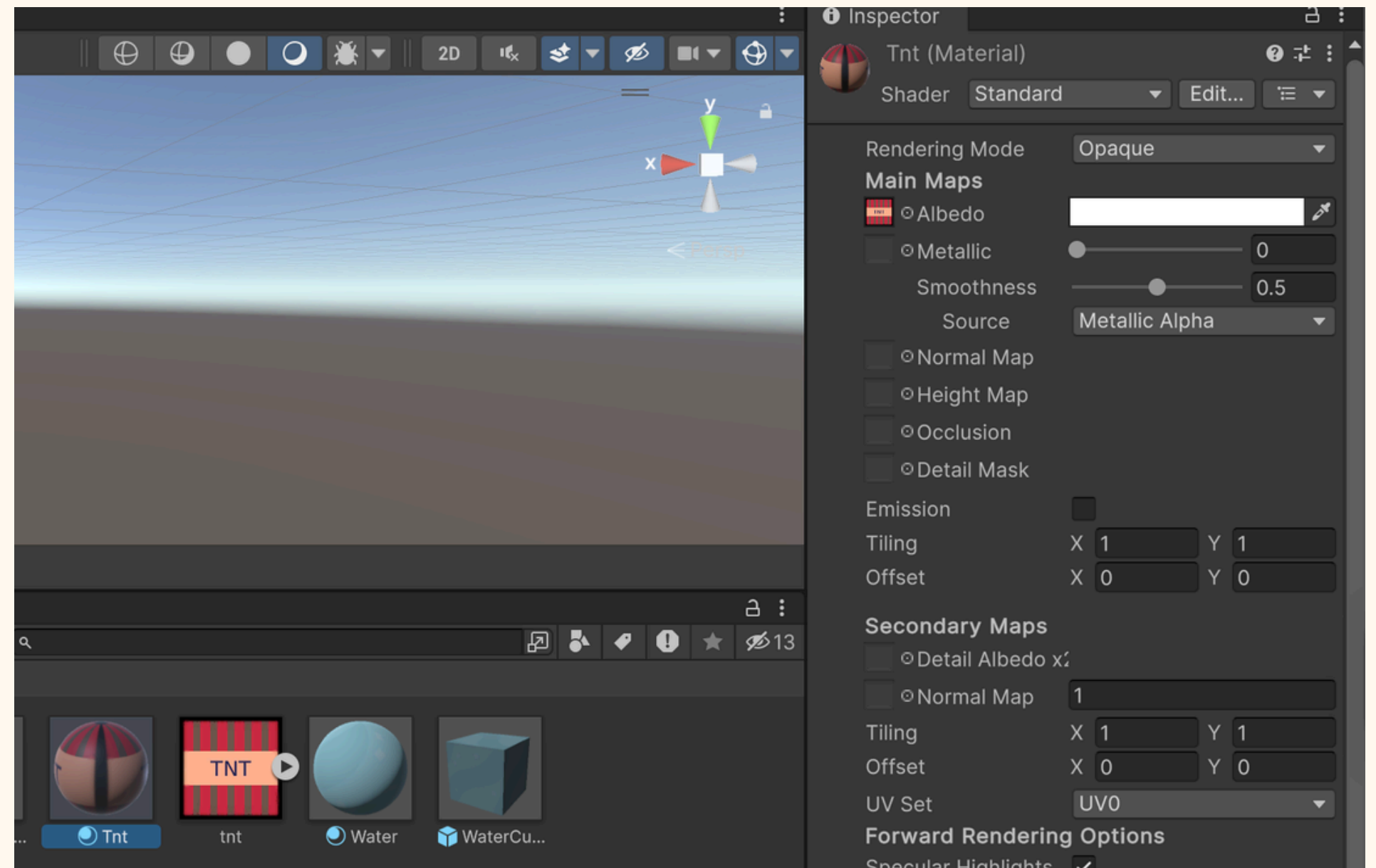
Comment ça marche ?

Quand la TNT explose, on va scanner tous les blocs dans un rayon de 3 blocs autour de la TNT, et les ajouter dans une liste pour ensuite tous les détruire en une fois.



**Pour cela, il nous faut 2 nouvelles fonctions dans le WorldManager.
Mais avant ça, il faut rajouter le bloc dans le jeu :)**

**Créez un nouveau matériau
et choisissez l'image de la
TNT comme texture**

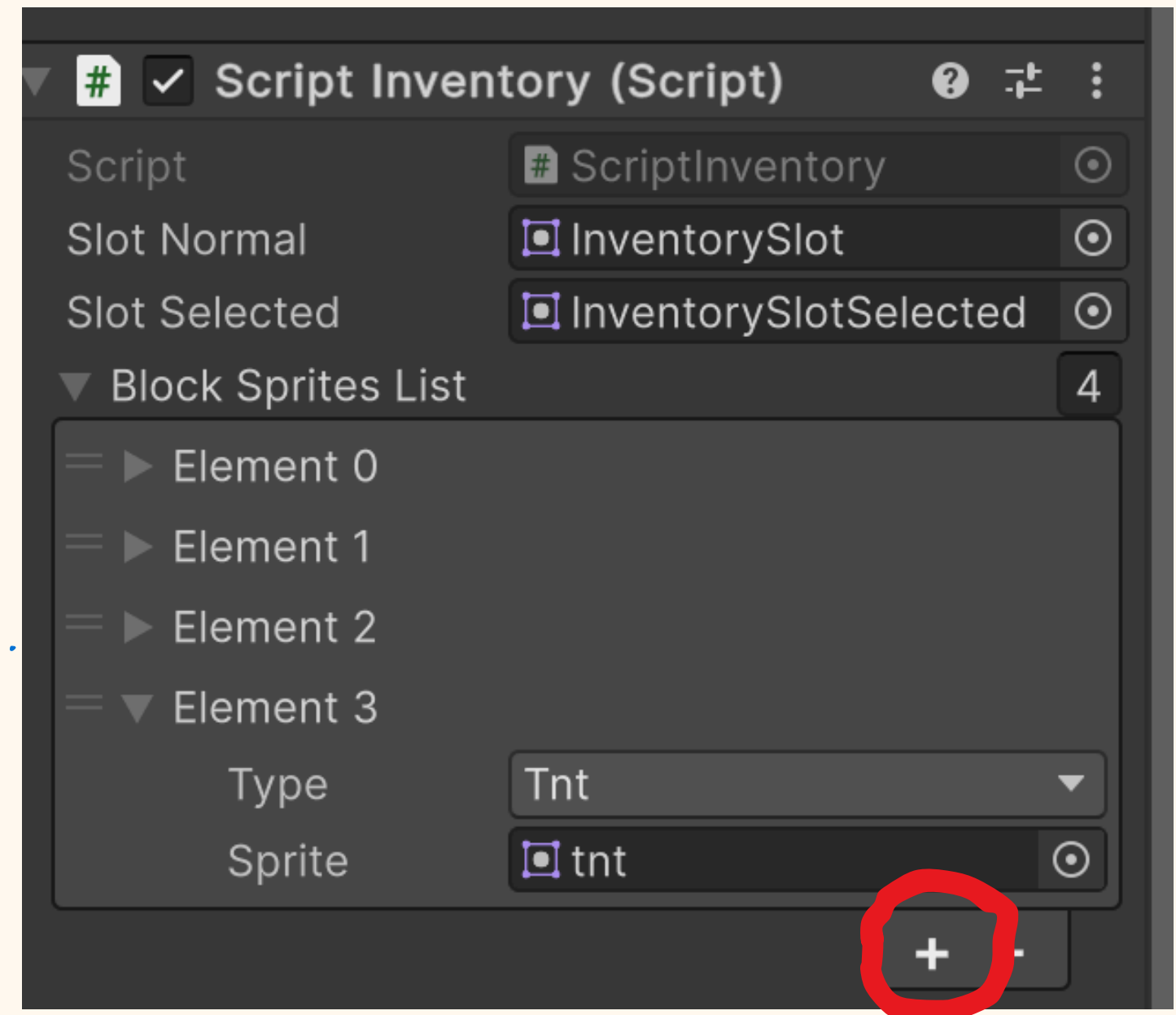


**Ajoutez la TNT
dans Blocktype et
dans la liste
hotbar_slots dans
le script de
l'inventaire**

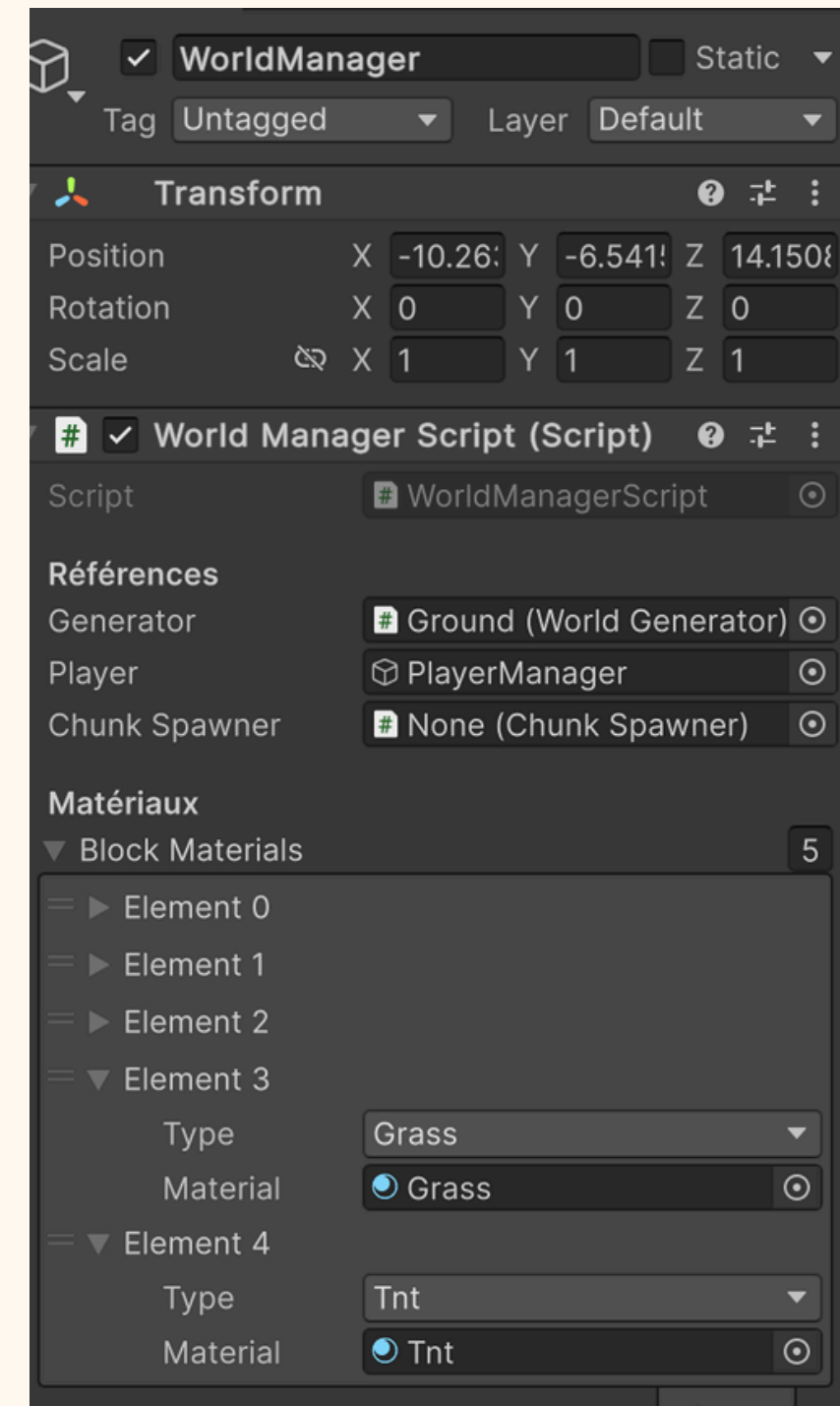
```
public enum BlockType
{
    2 references
    Air = 0, // vide
    3 references
    Dirt = 1, // terre
    4 references
    Stone = 2, // pierre
    3 references
    Grass = 3, // herbe
    2 references
    Tnt = 4,
}
```

```
5 references
private List<BlockType> hotbar_slots = new()
{
    BlockType.Dirt,
    BlockType.Grass,
    BlockType.Stone,
    BlockType.Tnt,
};
```

**Ajoutez la TNT
dans la liste de
l'inventaire**



Et dans le WorldManager



Ajoutez dans le WorldManagerScript

Cette fonction reçoit un Hashset (une liste où l'on supprime les doublons) "worldPos" qui contient tous les blocs à supprimer. On boucle sur chacun d'eux, on les supprime et on recharge le chunk

```
1 reference
public void DestroyMultipleBlocks(HashSet<Vector3Int> worldPos)
{
    foreach (Vector3Int block in worldPos)
    {
        if (!blocks.ContainsKey(block))
            continue;

        blocks.Remove(block);
        chunkSpawner.OnBlockDestroyed(block);
    }

    RebuildChunk();
}
```

Toujours dans worldManagerScript

```
public void Explode(Vector3 origin)
{
    int radius = 3;

    Vector3Int center = Vector3Int.FloorToInt(origin);

    HashSet<Vector3Int> blocksToDestroy = new HashSet<Vector3Int>();
```

La fonction explode se charge de scanner les blocs et de mettre la position des blocs dans une liste, prête à la passer à la fonction DeleteMultipleBlocks

On va définir :

- Le rayon de notre explosion
- La où se trouve notre TNT
- Et notre liste / Hashset avec les positions des blocks à supprimer !

On boucle ensuite sur toute les positions dans un rayon de 3 en X Y et Z.

Et on calcule la position de chaque bloc en ajoutant la position de base de la TNT.

```
for (int x = -radius; x <= radius; x++)
{
    for (int y = -radius; y <= radius; y++)
    {
        for (int z = -radius; z <= radius; z++)
        {
            Vector3Int offset = new Vector3Int(x, y, z);
            Vector3Int pos = center + offset;

            // sphere check (distance² <= radius²)
            if (offset.sqrMagnitude > radius * radius)
                continue;

            // only destroy existing blocks
            if (!blocks.ContainsKey(pos))
                continue;

            blocksToDestroy.Add(pos);
        }
    }
}
```

**Notre boucle forme un cube,
mais on veut une sphere donc
on skip les blocs en dehors de
la sphere**

```
for (int x = -radius; x <= radius; x++)  
{  
    for (int y = -radius; y <= radius; y++)  
    {  
        for (int z = -radius; z <= radius; z++)  
        {  
            Vector3Int offset = new Vector3Int(x, y, z);  
            Vector3Int pos = center + offset;  
  
            // sphere check (distance2 <= radius2)  
            if (offset.sqrMagnitude > radius * radius)  
                continue;  
  
            // only destroy existing blocks  
            if (!blocks.ContainsKey(pos))  
                continue;  
  
            blocksToDestroy.Add(pos);  
        }  
    }  
}
```

Si notre liste de blocs dans le monde ne contient pas de bloc à la position calculée, alors on passe au suivant. Et s'il y en a un alors on l'ajoute à la liste des blocs à supprimer

```
for (int x = -radius; x <= radius; x++)  
{  
    for (int y = -radius; y <= radius; y++)  
    {  
        for (int z = -radius; z <= radius; z++)  
        {  
            Vector3Int offset = new Vector3Int(x, y, z);  
            Vector3Int pos = center + offset;  
  
            // sphere check (distance² <= radius²)  
            if (offset.sqrMagnitude > radius * radius)  
                continue;  
  
            // only destroy existing blocks  
            if (!blocks.ContainsKey(pos))  
                continue;  
  
            blocksToDestroy.Add(pos);  
        }  
    }  
}
```

```
        blocksToDestroy.Add(pos);  
    }  
}  
  
DestroyMultipleBlocks(blocksToDestroy);  
}
```

**Et pour finir, on appelle
DestroyMultipleBlocks avec notre liste !**

Mais comment activer la TNT ?

On va activer la TNT avec la touche “e” pour se simplifier la vie.

Si on appuye sur “e”, on lance un RayCast pour savoir quel bloc on a touché

```
if (Input.GetKeyDown(KeyCode.E))
{
    RaycastHit? hit = GetHit();

    if (!hit.HasValue)
        return;

    Vector3Int block = GetHitBlockPosition(hit.Value);
    var p = Vector3Int.FloorToInt(block);

    if (worldManager.Blocks[p] == BlockType.Tnt)
    {
        worldManager.Explode((Vector3)block);
    }
}
```

**Et si c'est une TNT, alors elle
explose !**

**(C'est ici qu'on donne les
coordonnées "origin", pour
savoir où est l'explosion)**

```
if (Input.GetKeyDown(KeyCode.E))
{
    RaycastHit? hit = GetHit();

    if (!hit.HasValue)
        return;

    Vector3Int block = GetHitBlockPosition(hit.Value);
    var p = Vector3Int.FloorToInt(block);

    if (worldManager.Blocks[p] == BlockType.Tnt)
    {
        worldManager.Explode((Vector3)block);
    }
}
```

**Vous pouvez maintenant
faire disparaître des blocs !**

**Mais il manque un petit
quelque chose... des
particules de l'explosion !**

```
[Header("VFX")]
```

2 references

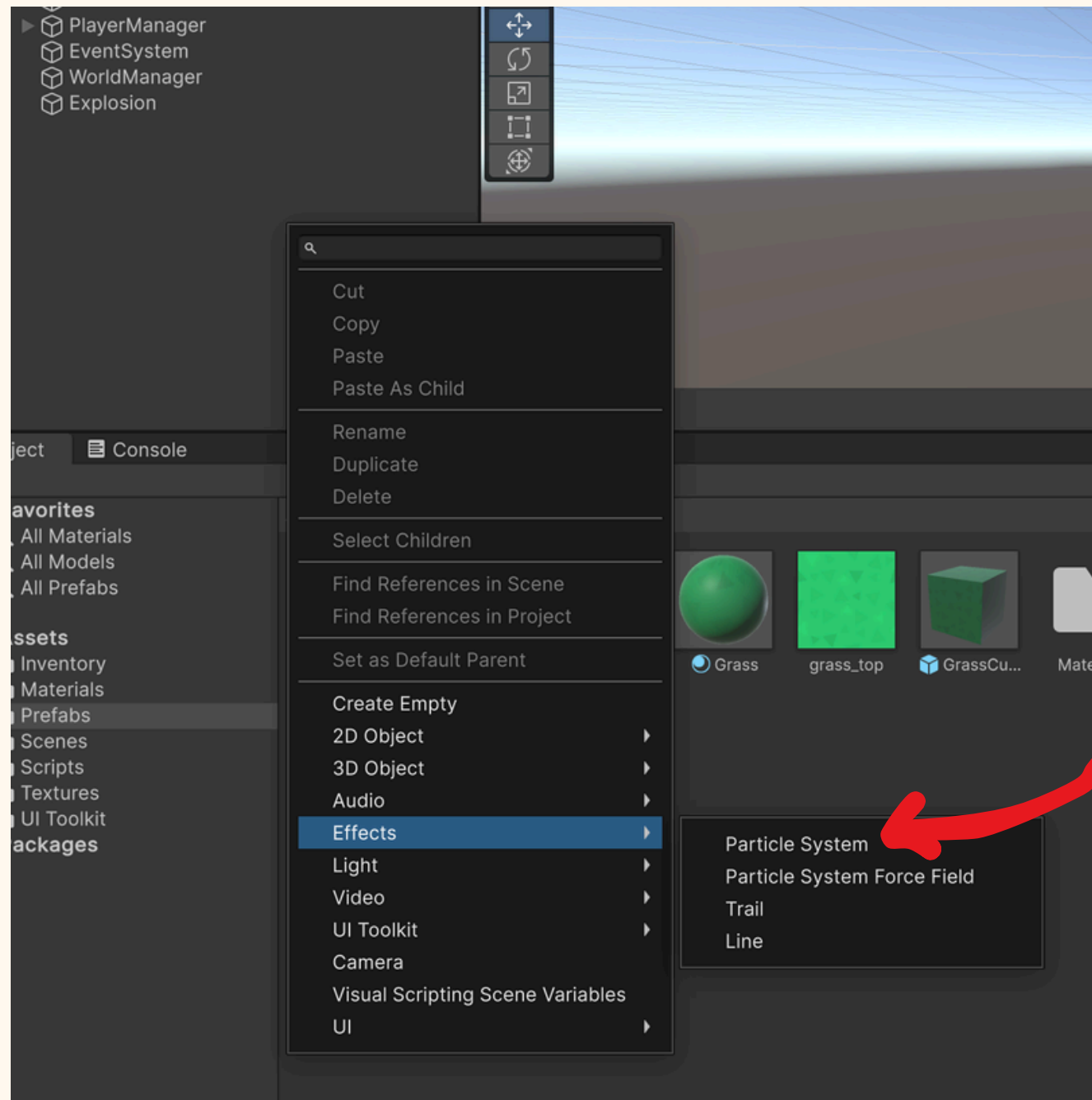
```
public ParticleSystem explosionPrefab;
```

**Ajoutez la variable explosionPrefab dans
notre WoldManagerScript.
Un VFX est un effet visuel (nos particules)**

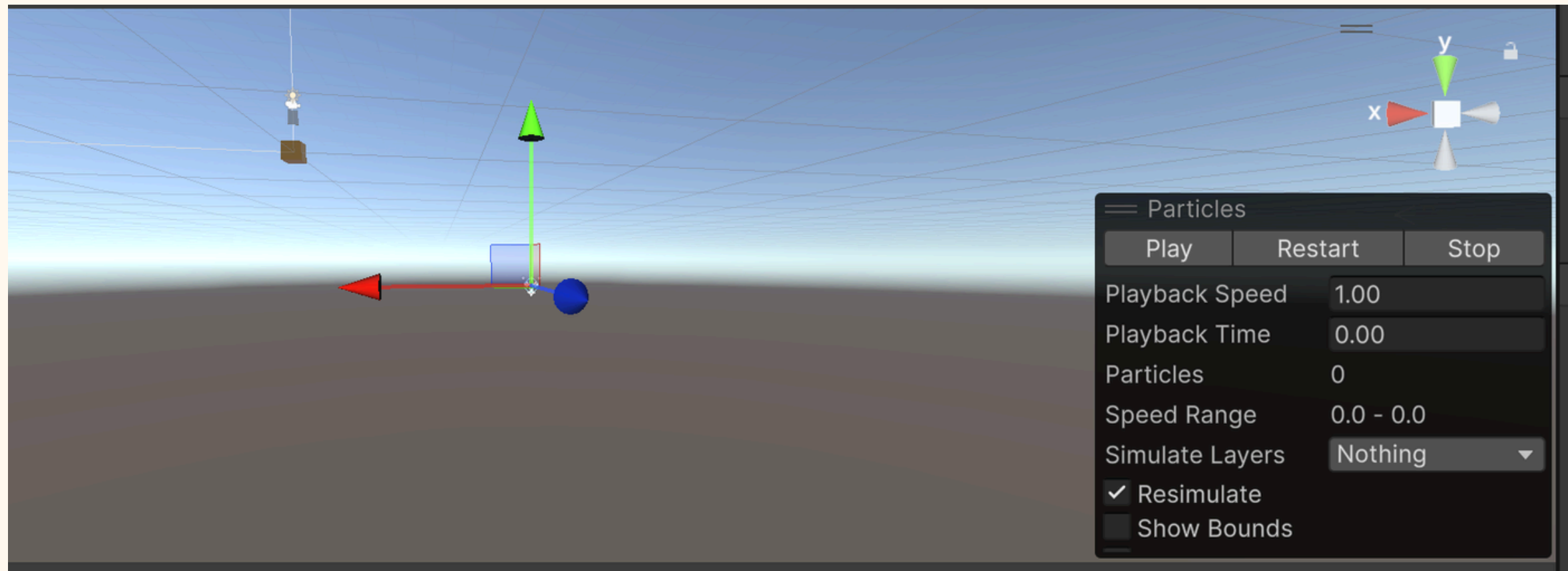
```
public void Explode(Vector3 origin)
{
    int radius = 3;

    if (explosionPrefab != null)
    {
        ParticleSystem vfx = Instantiate(explosionPrefab, origin, Quaternion.identity);
        vfx.Play();
        Destroy(vfx.gameObject, vfx.main.duration + vfx.main.startLifetime.constantMax);
    }
}
```

Ajoutez ces lignes dans Explode. Si explosionPrefab existe, alors on fait apparaître les particules sur notre bloc, puis on les supprime quand l'animation est finie.

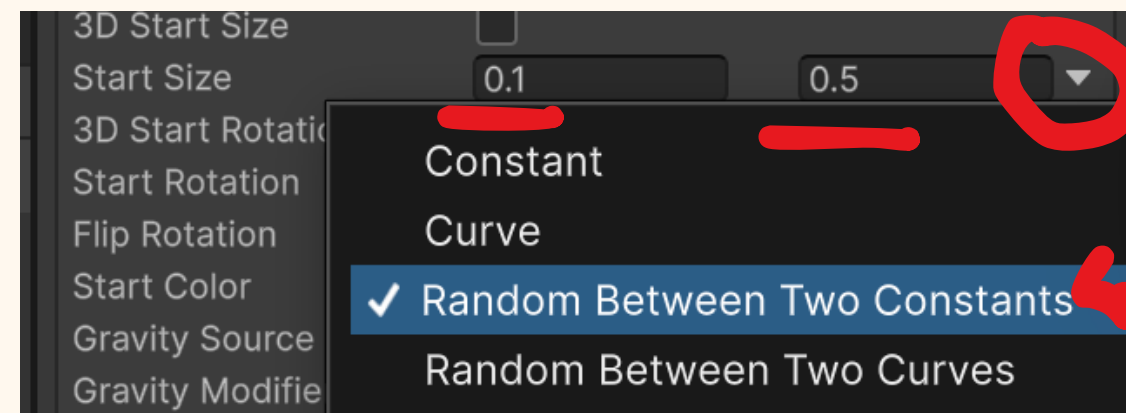


**Créez un générateur de particules
et faites en un prefab**

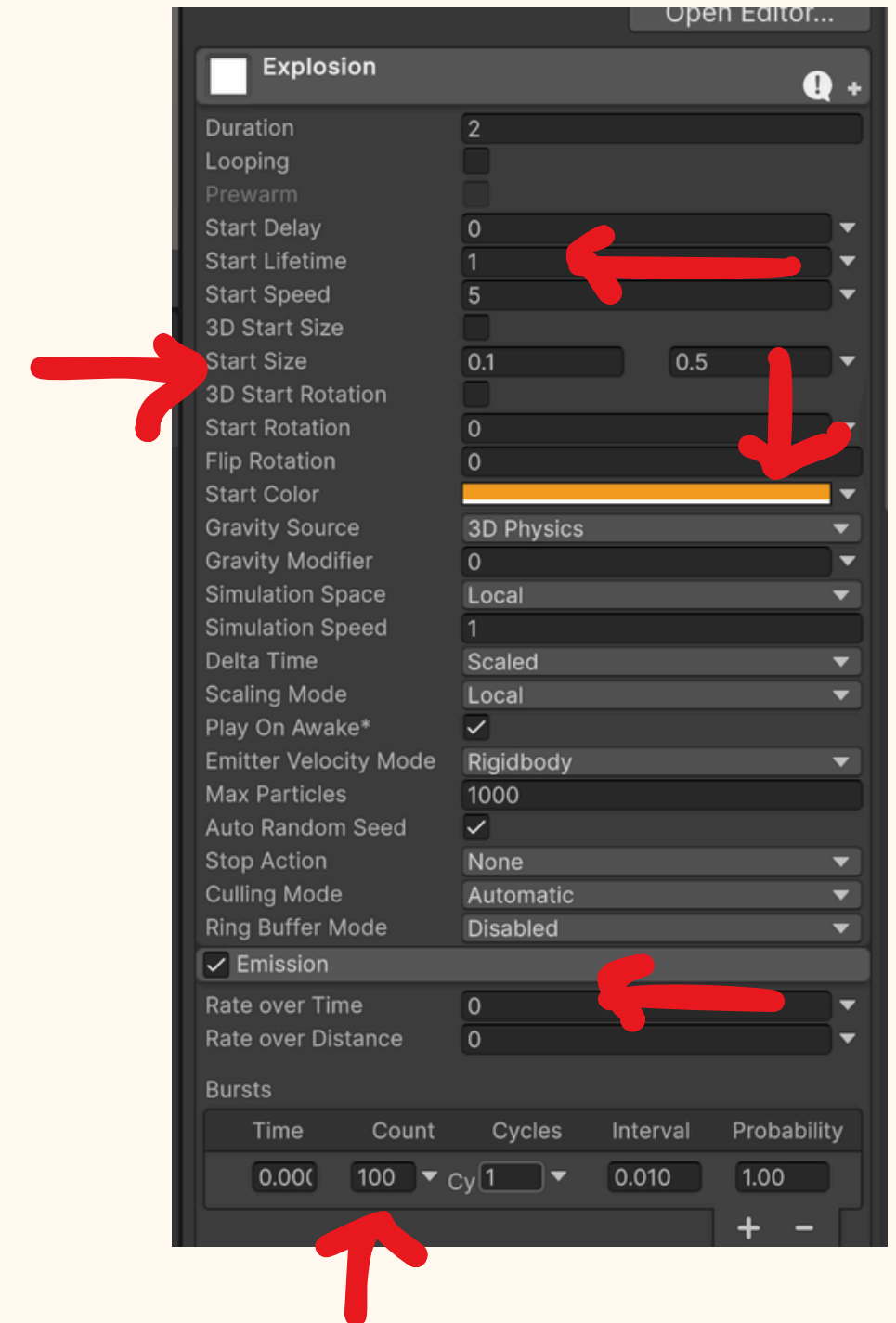


**Quand vous cliquez sur votre
générateur, vous pouvez appuyer
sur Play pour lancer l'animation**

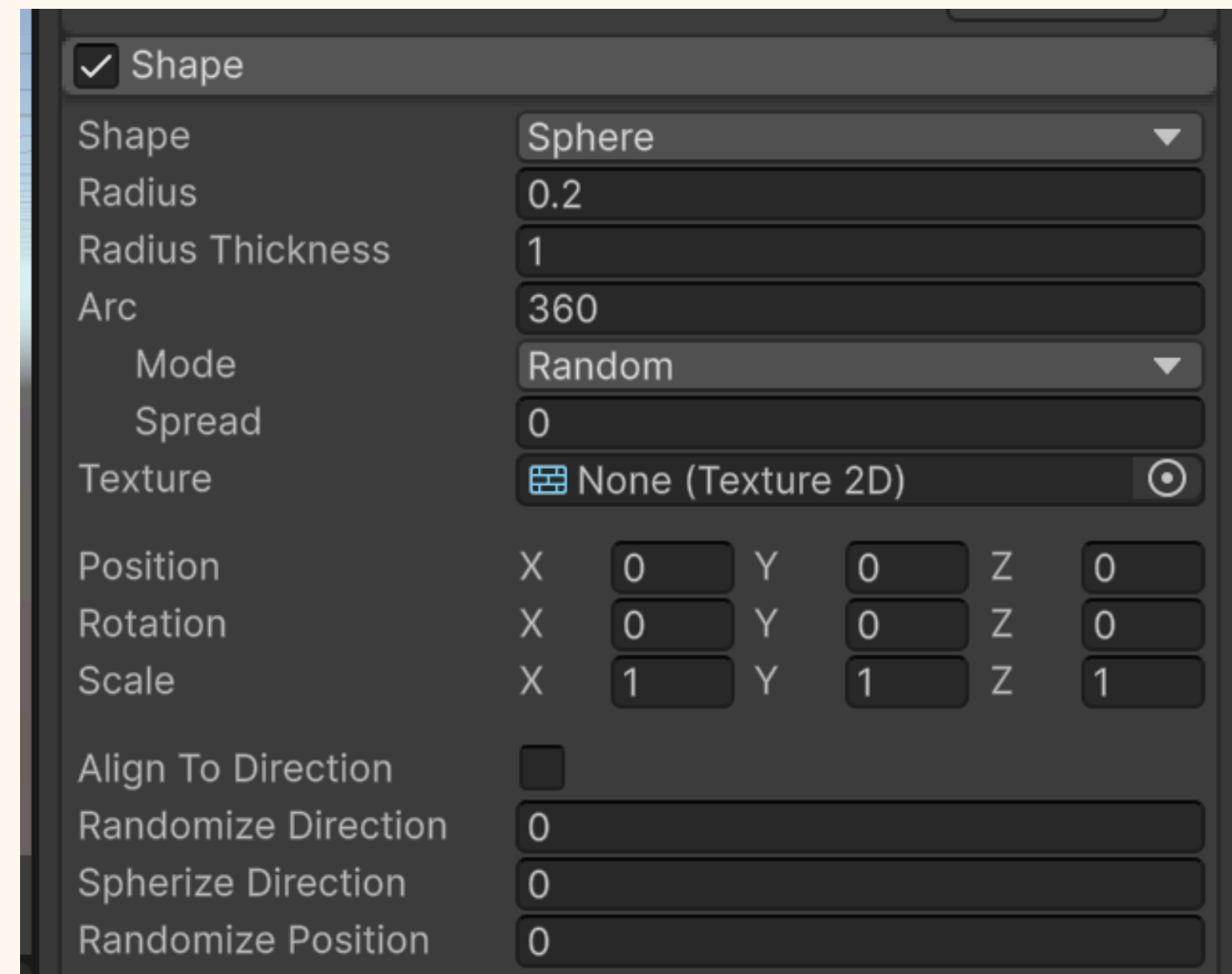
Voici des paramètres qui marchent bien, mais changez ce que vous voulez ;)

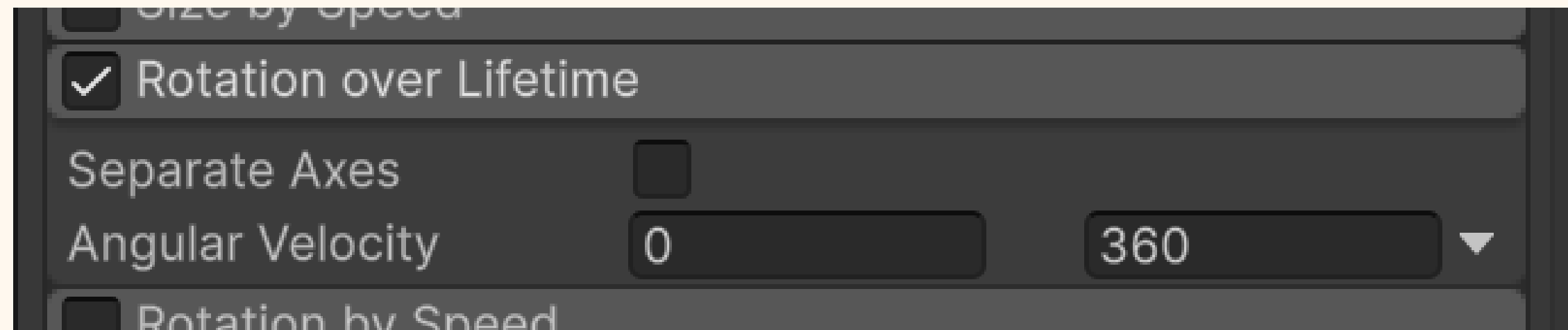


Pour le start size, on va choisir des tailles aleatoires pour nos particules



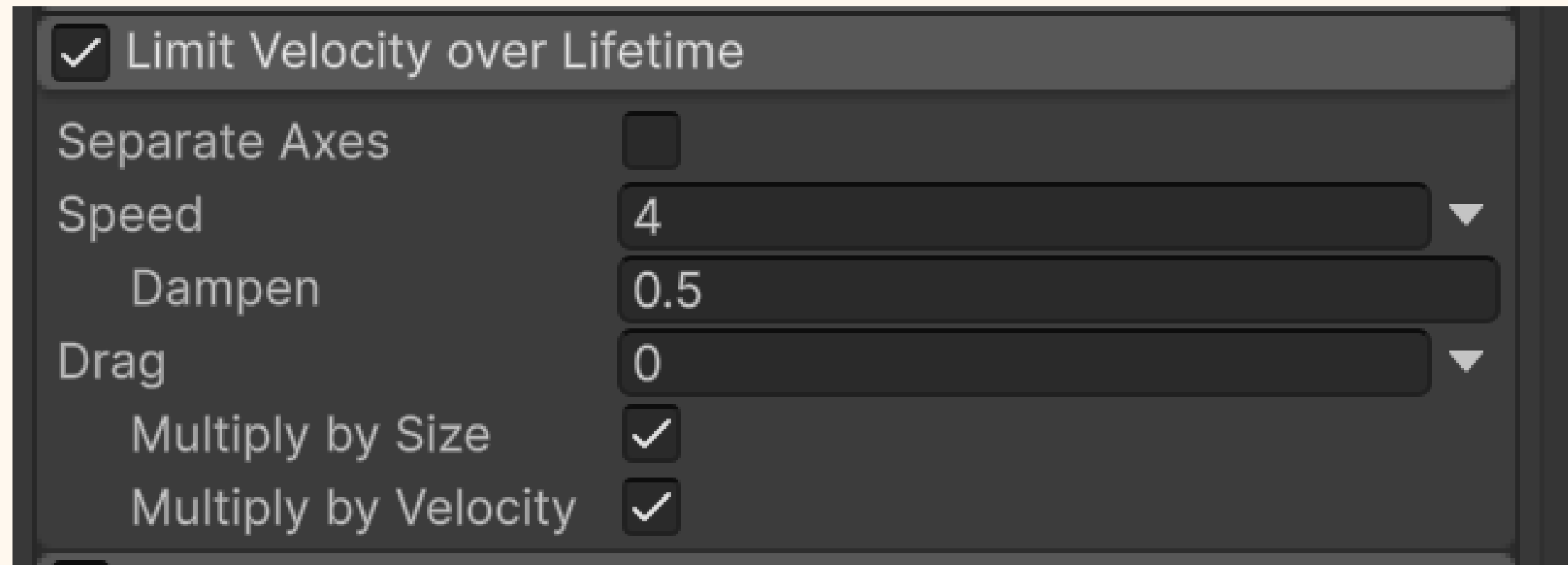
**Pour la forme, choisissez une
sphère, avec un petit radius au
départ. Elle vas s'agrandir car nos
particules bougent**





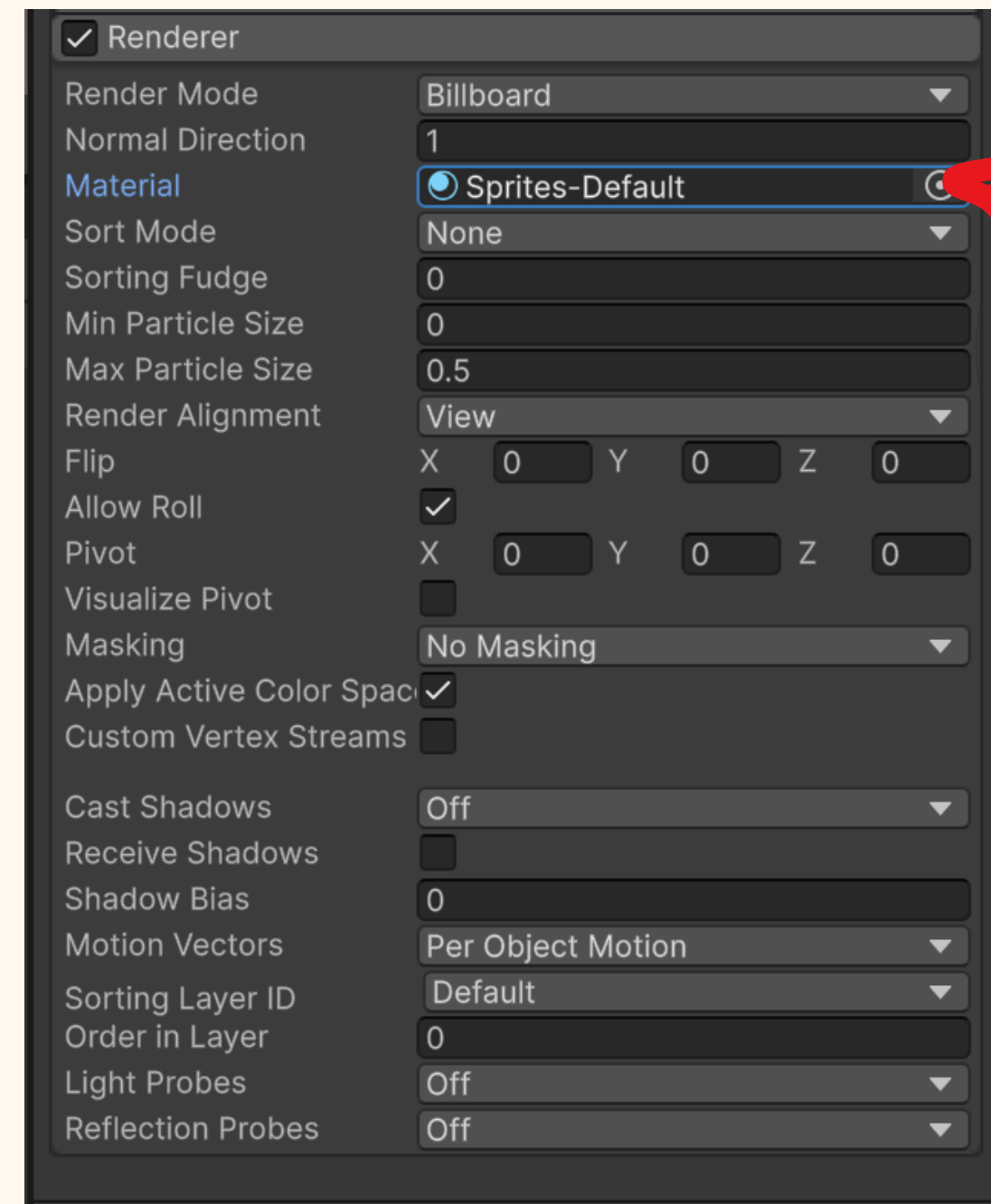
Rotation Over Lifetime permet à nos particules de tourner sur elles-mêmes pour rajouter de l'effet :)

N'oubliez pas de cliquer sur la flèche pour sélectionner deux variables

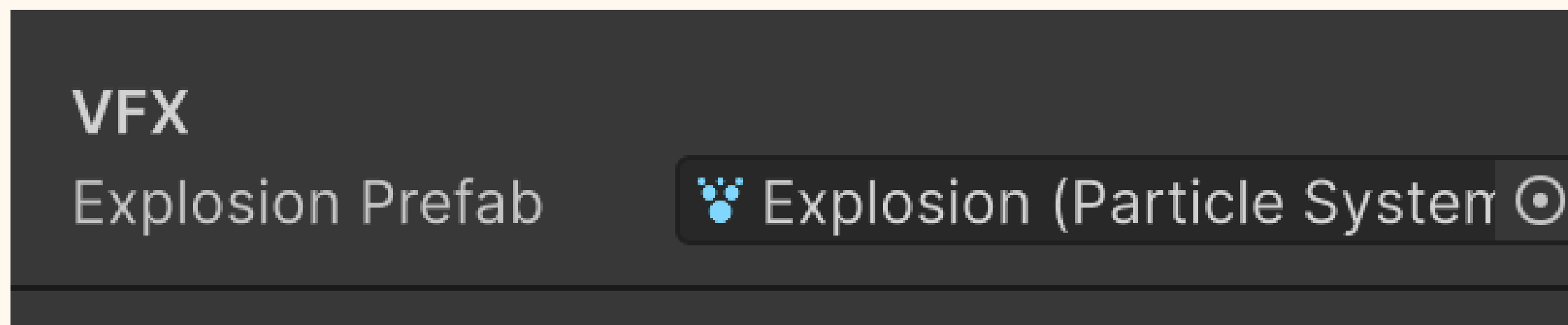


Limit Velocity Over Lifetime permet à nos
particules de ralentir plus le temps avance

**Dans Renderer, vous pouvez
modifier la forme des particules.
J'ai choisi le Sprite-Default qui est
un carré**



**Quand vous êtes content de votre
explosion, rajoutez votre prefab
dans le world Manager**



Et voilà